Lab Handout 1-G

Interceptor (Threat) Firmware: ISR-Driven Capture, RSSI Table, and Button-Gated PAUSE TX

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Implement the third radio's firmware: the interceptor ("Threat", Beacon ID `0xFF`) that passively captures every DATA, ACK, and PAUSE frame on the shared 906.5 MHz channel via an `RxDone` interrupt, queues frames in a ring buffer, and forwards them from the main loop to the Field-Programmable Gate Array (FPGA) over a 43-byte framed UART protocol. In parallel, the firmware maintains a per-Beacon-ID Received Signal Strength Indicator (RSSI) observation table that is updated on every received packet, and runs a button-gated PAUSE TX scheduler: while the user button (B1) on the Threat board is held, the firmware emits a PAUSE packet at ∼100 ms cadence addressed to whichever non-stale Beacon ID currently has the highest RSSI, with a 500 ms hold-off duration in the payload. This is the software side of the Session 4 deliverable.

2 Prerequisites

- Lab Handouts 1-A, 1-B, 1-C, 1-D complete.
- Lab Handout 1-F (cooperative target firmware) at least partially complete — the two Targets must be running ping-pong as a traffic source for Threat capture validation. Steps 1–6 of LH1-F (DATA/ACK ping-pong without the PAUSE listener) are the minimum required.
- No external USB-to-UART adapter is required. Step 7's behavioral validation runs entirely against the Threat's USART2 console (the existing ST-Link COM port). Frame-level validation moves to LH2-A and LH2-B where the FPGA's onboard USB-UART captures the byte stream directly.

USART3 is not on the ST-Link USB Port

The Nucleo's single USB-B port is the ST-Link Virtual COM Port, which is wired internally to `USART2` only (`PA2`/`PA3`). `USART3` (`PC4`/`PC5` on this project) lands on the Morpho header pins and is *not* routed to that USB port. The Threat will print its `printf` debug log (boot banner, per-RX rows, `PAUSE -> ID=...`) on `USART2` as before, but the 43-byte binary forwarding frames only appear on the `PC4` pin — you must jumper that pin to the Nexys A7 PMOD (LH1-H) before any FPGA or host can see them.

[Figure placeholder]

`LH1G_isr_mainloop_dataflow.png`

Threat firmware data flow: `DIO0` → `EXTI` → Interrupt Service Routine (ISR) (FIFO read, RSSI/Signal-to-Noise Ratio (SNR), timestamp, RSSI-table update) → ring buffer → main loop (frame build, Cyclic Redundancy Check (CRC), UART DMA out to FPGA). A second main-loop poll of `B1` drives the PAUSE TX scheduler at ∼100 ms cadence whenever the button is held; PAUSE TX shares the radio with RX in half-duplex.

Figure 1: Threat firmware data flow: `DIO0` → `EXTI` → ISR (FIFO read, RSSI/SNR, timestamp, RSSI-table update) → ring buffer → main loop (frame build, CRC, UART DMA out to FPGA). A second main-loop poll of `B1` drives the PAUSE TX scheduler at ∼100 ms cadence whenever the button is held; PAUSE TX shares the radio with RX in half-duplex.

3 Forwarding Frame Format

The interceptor wraps each received radio packet (DATA, ACK, or PAUSE — the FPGA does not care which) in a framed UART message. Total 43 bytes. This layout is the canonical spec from the Module 1 lesson plan (Table 6) and is consumed verbatim by the Module 2 parser; do not reorder fields.

Offset	Field	Length
0	Start delimiter (`0x7E`)	1
1	Frame length (always `0x29` = 41)	1
2–5	RX timestamp (uint32, LE, ms)	4
6	RSSI (int8, dBm, two's complement)	1
7	SNR (int8, dB, 0.25 dB resolution)	1
8–9	Frequency error (int16, LE, Hz)	2
10–41	Radio packet (32 bytes: DATA, ACK, or PAUSE)	32
42	Frame CRC-8/ITU (poly `0x07`) over bytes 1–41	1

Length-Field Rule

The frame-length byte is defined as the count of bytes *excluding the start delimiter and the trailing CRC*. For a 43-byte frame, byte[1] must equal `0x29` (41 decimal). The radio packet's embedded CRC-16/CCITT (radio packet bytes 30–31) lands at frame offsets 40–41 and is *not* the frame CRC; the frame CRC-8 is at offset 42.

[Figure placeholder]

`LH1G_frame_layout.png`

43-byte forwarding frame layout: delimiter, length, metadata (timestamp, RSSI, SNR, freq. error), embedded 32-byte radio packet, and trailing CRC-8 at byte 42.

Figure 2: 43-byte forwarding frame layout: delimiter, length, metadata (timestamp, RSSI, SNR, freq. error), embedded 32-byte radio packet, and trailing CRC-8 at byte 42.

4 Architecture

4.1 ISR path

1. DIO0 rising edge triggers an `EXTI` interrupt.
2. ISR reads `RegIrqFlags`. If `RxDone` is set and `PayloadCrcError` is clear:
   (a) Read `RegFifoRxCurrentAddr`, set `RegFifoAddrPtr`, read 32 bytes from `RegFifo`.
   (b) Read `RegPktRssiValue`, `RegPktSnrValue`, and `RegFei*` for the frequency error.
   (c) Stamp the current `ms_count`.
   (d) Push a struct (`intercepted_pkt_t`) containing these fields into a *lock-free single-producer/single-consumer ring buffer* (producer = ISR, consumer = main loop).
   (e) Update the per-Beacon-ID RSSI table: `rssi_table[sender_id] = {rssi_dbm, ms_count}`, where `sender_id` is the radio packet's byte[2].
   (f) Clear `RegIrqFlags` by writing `0xFF`.

4.2 Main-loop path

1. Check ring buffer head vs. tail. If not empty, dequeue one packet.
2. Build the 43-byte forwarding frame.
3. Compute the frame CRC-8/ITU (poly `0x07`, init `0x00`) over bytes 1–41 and place it in byte 42. The embedded radio-packet CRC-16 (already in the 32-byte payload) is copied through unchanged at frame offsets 40–41.
4. Transmit the entire 43-byte frame over `USART3` (or whichever Universal Asynchronous Receiver/Transmitter (UART) is wired to the Nexys A7) using `HAL_UART_Transmit_DMA`.
5. Increment a `frames_forwarded` counter.
6. Poll the user button (B1, `PC13`, active-low). If pressed, run the PAUSE TX scheduler (Step 5 below).

[Figure placeholder]

`LH1G_ring_buffer_diagram.png`

Single-producer/single-consumer ring buffer: ISR advances `head`, main loop advances `tail`. A `__DMB()` barrier orders the payload write against the head increment.

Figure 3: Single-producer/single-consumer ring buffer: ISR advances `head`, main loop advances `tail`. A `__DMB()` barrier orders the payload write against the head increment.

4.3 Ring buffer sizing

A 16-entry ring (≈640 bytes of intercepted-packet structs) accommodates roughly 16 back-to-back receptions without overflow. With Target A and Target B in 1 s ping-pong (effectively two captured frames per second) plus PAUSE bursts at 10 Hz while the button is held, a 16-entry ring gives ∼1.5 s of margin against a stalled main loop — comfortable.

4.4 RSSI observation table

A 256-entry array indexed by Beacon ID, with each entry containing the most recent RSSI (in dBm) and the `ms_count` at which that observation was captured. The `last_seen_ms` field is what makes the table self-pruning: an entry is considered *stale* (and excluded from highest-RSSI lookups) if `ms_count - last_seen_ms` exceeds a configurable threshold (5 s default). Without this discipline, a Target that briefly transmitted hours ago and then went quiet would still appear in the table at its old RSSI value and could become the PAUSE target by mistake — a silent integration fault that the Module 5 failure-mode catalog records as the canonical depth-2 entry tied to Module 1's staleness threshold choice.

4.5 PAUSE TX scheduler

A button-gated, main-loop-driven scheduler. Pseudocode:

1. If `HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET` (button pressed, active-low) and `ms_count - last_pause_tx_ms` $\geq$ `100`:

   (a) Scan the RSSI table. Pick the non-stale Beacon ID with the highest RSSI.

   (b) If no non-stale entry exists, log `"no PAUSE target available"` and skip.

   (c) Build a 32-byte PAUSE packet via `build_packet(buf, BEACON_ID_THREAT, pause_seq, PKT_TYPE_PAUSE, MOD_KIND_LORA)`, then overwrite `buf[13] = chosen_id` and `buf[14] = 5` (5 units × 100 ms = 500 ms hold-off), then *recompute* the CRC-16 over bytes 0–29 (the payload bytes changed).

   (d) Switch the radio out of `RXCONTINUOUS` (the receive path is unavoidably interrupted for the ∼50 ms PAUSE airtime; this is the documented half-duplex cost).

(e) TX the PAUSE packet.

(f) Switch back to `RXCONTINUOUS`.

(g) Log on USART2: `"PAUSE -> ID=%02X (RSSI=%d dBm, seq=%lu)"`.

(h) `last_pause_tx_ms = ms_count`; `pause_seq++`.

5 Procedure

5.1 Step 1 — Open the pre-existing Threat project

The `Nucleo_RFM95W_Threat` project was created up front in Lab Handout 1-C, Step 1, with the same baseline peripheral configuration as Target A and Target B (SPI1, USART2, USART3 with TX DMA, TIM2 1 ms tick, `RFM_CS`/`RFM_RST`/`RFM_DIO0` GPIO labels, EXTI on PA8 with the EXTI[9:5] NVIC line enabled). The user button B1 on `PC13` is configured as a GPIO input by default on every Nucleo-L476RG project. **Do not copy, rename, or move the project, and do not run CubeMX again.**

1. Open `Nucleo_RFM95W_Threat` in its own VS Code window (separate from the `TargetA` and `TargetB` windows).
2. Press `F7` to confirm the project still builds.
3. In the integrated terminal, confirm the build is using the LH1-C-generated peripheral init by inspecting the build log for references to `MX_SPI1_Init`, `MX_USART2_UART_Init`, `MX_USART3_UART_Init`, `MX_TIM2_Init`, and `MX_DMA_Init`. All five must be present.

5.2 Step 2 — Ring buffer and intercepted-packet struct

Add the ring buffer definitions and the explicit `push`/`pop` primitives to `main.c`. The push runs in ISR context; the pop runs in main-loop context. They are coupled by a single `__DMB()` barrier on the producer side that orders the payload write before the head increment.

```c
/* --- USER CODE BEGIN PTD --- */
typedef struct {
    uint8_t  raw[32];          /* 32-byte radio packet as received */
    uint32_t rx_timestamp_ms;  /* ms tick at RxDone */
    int8_t   rssi_dbm;
    int8_t   snr_q025;         /* 0.25 dB resolution */
    int16_t  freq_error_hz;
} intercepted_pkt_t;
/* --- USER CODE END PTD --- */

/* --- USER CODE BEGIN PV --- */
#define RING_SIZE 16u                          /* must be power of two */
#define RING_MASK (RING_SIZE - 1u)

static intercepted_pkt_t ring[RING_SIZE];
volatile uint32_t ring_head = 0;               /* writer (ISR)         */
volatile uint32_t ring_tail = 0;               /* reader (main loop)   */
volatile uint32_t dropped   = 0;
/* --- USER CODE END PV --- */
```

Listing 1: Add inside `/* USER CODE BEGIN PTD */` (typedef) and `/* USER CODE BEGIN PV */` (storage).

```c
/* Producer: called from the DIO0 ISR. Returns 1 on success, 0 on overflow
    . */
static inline int ring_push(const intercepted_pkt_t *src) {
    uint32_t head = ring_head;
```

```c
    uint32_t next = (head + 1u) & RING_MASK;
    if (next == ring_tail) {
        dropped++;
        return 0;
    }
    ring[head] = *src;      /* full struct copy, ~46 bytes */
    __DMB();                /* order payload write before head update */
    ring_head = next;
    return 1;
}

/* Consumer: called from the main loop. Returns 1 if a packet was dequeued. */
static inline int ring_pop(intercepted_pkt_t *dst) {
    uint32_t tail = ring_tail;
    if (tail == ring_head) return 0;      /* empty */
    *dst = ring[tail];
    __DMB();
    ring_tail = (tail + 1u) & RING_MASK;
    return 1;
}
```

Listing 2: Place inside `/* USER CODE BEGIN 4 */`. Single-producer / single-consumer; no critical section needed because head and tail are written by exactly one context each.

5.3 Step 3 — RSSI observation table

1. Define the table:

   ```c
   typedef struct {
       int8_t   rssi_dbm;
       uint32_t last_seen_ms;
   } rssi_obs_t;

   #define RSSI_STALE_MS 5000
   static volatile rssi_obs_t rssi_table[256];
   ```

2. In `main()` after `lora_init()`, initialize:

   ```c
   for (int i = 0; i < 256; i++) {
       rssi_table[i].rssi_dbm = -128;
       rssi_table[i].last_seen_ms = 0;
   }
   ```

3. Update inside the ISR (after the FIFO read, before the ring `push`):

   ```c
   uint8_t sender = pkt_buf.raw[2];
   rssi_table[sender].rssi_dbm     = pkt_buf.rssi_dbm;
   rssi_table[sender].last_seen_ms = pkt_buf.rx_timestamp_ms;
   ```

4. Implement the highest-RSSI lookup as a main-loop helper:

```
int find_highest_rssi_id(uint8_t *out_id) {
    int8_t best_rssi = -128;
    int best_id = -1;
    uint32_t now = ms_count;
    for (int i = 1; i < 256; i++) {  /* skip 0x00 = invalid */
        if (i == BEACON_ID_THREAT) continue;
        if (rssi_table[i].last_seen_ms == 0) continue;
        if ((now - rssi_table[i].last_seen_ms) > RSSI_STALE_MS) continue;
        if (rssi_table[i].rssi_dbm > best_rssi) {
            best_rssi = rssi_table[i].rssi_dbm;
            best_id = i;
        }
    }
    if (best_id < 0) return -1;
    *out_id = (uint8_t)best_id;
    return 0;
}
```

5.4 Step 4 — DIO0 ISR

The Threat's ISR differs from the cooperative-target ISR (LH1-F Step 3) in one important way: it does *not* act on the packet contents, it only stamps and queues. All policy (forwarding, PAUSE-target selection, USART2 logging) happens in the main loop. This keeps the ISR bounded to a few microseconds and prevents RF packet drops while the host UART drains.

Add the following inside `/* USER CODE BEGIN 4 */`, after the ring buffer code from Step 2:

```
/* SX1276 LoRa-mode frequency-error registers: 20-bit signed in
 * RegFeiMsb (0x28, low nibble), RegFeiMid (0x29), RegFeiLsb (0x2A).
 * (Registers 0x1D..0x1F hold FSK/OOK FEI; in LoRa mode they are
 * RegModemConfig1/2 and must NOT be read for FEI.)
 *
 * fei_hz = raw * F_XOSC / 2^24 * (BW / 500 kHz).
 * For F_XOSC=32 MHz, BW=125 kHz: fei_hz = raw * 32e6 / 2^26
 *                                       = (raw * 32000000) >> 26.
 * The result is small enough to fit in int16 except at extreme offsets,
 * so we clip to the int16 range for the forwarding-frame field.
 */
static int16_t read_fei_hz(void) {
    uint8_t h = rfm_read_reg(0x28u);
    uint8_t m = rfm_read_reg(0x29u);
    uint8_t l = rfm_read_reg(0x2Au);
    int32_t raw = ((int32_t)(h & 0x0Fu) << 16)
                |  ((int32_t)m            <<  8)
                |   (int32_t)l;
    if (raw & 0x00080000) raw |= 0xFFF00000;     /* sign-extend 20-bit */
    int32_t hz = (int32_t)(((int64_t)raw * 32000000) >> 26);
    if (hz >  32767) hz =  32767;
    if (hz < -32768) hz = -32768;
    return (int16_t)hz;
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
```

```c
    if (GPIO_Pin != RFM_DIO0_Pin) return;

    uint8_t irq = rfm_read_reg(0x12u);
    if ((irq & 0x40u) == 0u) {                    /* not RxDone */
        rfm_write_reg(0x12u, 0xFFu);
        return;
    }
    if (irq & 0x20u) {                            /* PayloadCrcError */
        rfm_write_reg(0x12u, 0xFFu);
        return;
    }

    intercepted_pkt_t pkt;
    uint8_t rx_addr = rfm_read_reg(0x10u);        /* RegFifoRxCurrentAddr */
    rfm_write_reg(0x0Du, rx_addr);                /* RegFifoAddrPtr */
    for (int i = 0; i < 32; i++) pkt.raw[i] = rfm_read_reg(0x00u);

    pkt.rx_timestamp_ms = ms_count;
    pkt.rssi_dbm        = (int8_t)((int16_t)rfm_read_reg(0x1Au) - 157);
    pkt.snr_q025        = (int8_t)rfm_read_reg(0x19u);
    pkt.freq_error_hz   = read_fei_hz();

    /* RSSI table update (Threat's job; not done in cooperative-target ISR) */
    uint8_t sender = pkt.raw[2];
    rssi_table[sender].rssi_dbm     = pkt.rssi_dbm;
    rssi_table[sender].last_seen_ms = pkt.rx_timestamp_ms;

    (void)ring_push(&pkt);                        /* dropped++ on overflow */
    rfm_write_reg(0x12u, 0xFFu);                  /* clear all IRQ flags */
}
```

Listing 3: (LoRa-mode addresses `0x28..0x2A`) for the 20-bit signed frequency error and converts to int16 Hz.]`HAL_GPIO_EXTI_Callback` for the Threat. Stack-local struct, no printf, no main-loop policy. Reads RegFei[Msb/Mid/Lsb] (LoRa-mode addresses `0x28..0x2A`) for the 20-bit signed frequency error and converts to int16 Hz.

ISR-side SPI traffic is a real budget

The ISR above issues ∼40 SPI byte transactions (32 FIFO bytes + RegIrqFlags + RegFifoRxCurrentAddr + RegFifoAddrPtr + RegPktRssiValue + RegPktSnrValue + 3 FEI registers + IRQ clear). At the LH1-C SPI prescaler of 32 (= 2.5 MHz on a 80 MHz APB2), each byte takes ∼3.2 μs, so the ISR runs ∼130 μs. This is fine for a 2 Hz ping-pong source but eats noticeable margin if you ever crank the source rate above ∼1 kHz; if you do, switch to a SPI-DMA burst read of the 32-byte FIFO.

5.5 Step 5 — PAUSE TX scheduler in the main loop

The main loop has two responsibilities: drain the ring buffer (frame builder + UART forwarder) and poll the button (PAUSE TX scheduler). They share the radio in half-duplex, so the scheduler

temporarily switches the radio out of `RXCONTINUOUS` for each PAUSE TX.

1. Add the scheduler state near the top of the main loop:

```
uint32_t last_pause_tx_ms = 0;
uint16_t pause_seq = 0;
```

2. Inside the `while(1)` loop, add the scheduler logic after the ring-buffer drain:

```
/* --- Drain ring buffer and forward frames to FPGA --- */
intercepted_pkt_t pkt;
while (ring_pop(&pkt)) {
    uint8_t frame[43];
    build_forwarding_frame(frame, &pkt);  /* per Section "Forwarding Frame" */
    HAL_UART_Transmit_DMA(&huart3, frame, 43);
    /* wait for DMA complete or use a tx_busy flag */

    /* Optional debug print */
    uint8_t pkt_type = pkt.raw[6] & 0x03;
    printf("RX sender=%02X type=%u rssi=%d snr=%d\r\n",
           pkt.raw[2], pkt_type, (int)pkt.rssi_dbm,
           (int)(pkt.snr_q025 / 4));
}

/* --- PAUSE TX scheduler --- */
if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET) {
    if ((ms_count - last_pause_tx_ms) >= 100) {
        uint8_t target_id;
        if (find_highest_rssi_id(&target_id) == 0) {
            /* Build PAUSE packet */
            uint8_t pkt_buf[32];
            build_packet(pkt_buf, BEACON_ID_THREAT, pause_seq,
                         PKT_TYPE_PAUSE, MOD_KIND_LORA);
            pkt_buf[13] = target_id;
            pkt_buf[14] = 5;  /* 5 * 100 ms = 500 ms hold-off */
            for (int i = 15; i < 30; i++) pkt_buf[i] = 0xAA;
            /* Recompute CRC-16 over bytes 0-29 */
            uint16_t crc = crc16_ccitt(pkt_buf, 30);
            pkt_buf[30] = (uint8_t)(crc >> 8);
            pkt_buf[31] = (uint8_t)(crc);

            /* Switch to STANDBY, TX the PAUSE, then back to RX */
            rfm_set_mode_standby();
            rfm_write_reg(0x0D, rfm_read_reg(0x0E));
            for (int i = 0; i < 32; i++) rfm_write_reg(0x00, pkt_buf[i]);
            rfm_write_reg(0x22, 32);
            rfm_write_reg(0x12, 0xFF);
            rfm_write_reg(0x01, 0x83);  /* TX */
```

```
                /* Wait for TxDone with timeout */
                uint32_t tx_deadline = ms_count + 200;
                while (ms_count < tx_deadline
                       && (rfm_read_reg(0x12) & 0x08) == 0) {
                    /* spin */
                }
                rfm_write_reg(0x12, 0xFF);
                rfm_set_mode_rx();

                int8_t rssi = rssi_table[target_id].rssi_dbm;
                printf("PAUSE -> ID=%02X (RSSI=%d dBm, seq=%lu)\r\n",
                       target_id, (int)rssi, (unsigned long)pause_seq);

                last_pause_tx_ms = ms_count;
                pause_seq++;
            } else {
                /* No non-stale targets in the table */
                if ((ms_count - last_pause_tx_ms) >= 1000) {
                    printf("no PAUSE target available\r\n");
                    last_pause_tx_ms = ms_count;
                }
            }
        }
    }
```

5.6 Step 5b — Periodic RSSI snapshot (live target diagnostics)

The PAUSE TX path only logs when it actually emits a PAUSE, so if the button is not held — or if the scheduler reports `"no PAUSE target available"` — there is no visibility into *why*: which Beacon IDs the Threat is currently hearing, at what RSSI, and which one would be selected if the button were pressed right now. Without that, common field scenarios (one target moves out of range, both targets briefly fall stale, one target dominates the other by 20 dB) are invisible in the serial log.

Add a 1 Hz snapshot to the main loop that dumps every Beacon ID ever observed (with current RSSI and elapsed time since last reception) and the result of an unconditional `find_highest_rssi_id()` call. Declare a tracking timestamp alongside the Step 5 scheduler state:

```
uint32_t last_stats_ms = HAL_GetTick();   /* near last_pause_tx_ms */
```

Then add the snapshot block at the bottom of the main `while(1)` loop, gated by a 1-second window:

```
/* --- Periodic stats + per-ID RSSI snapshot (every 1 s) --- */
if ((HAL_GetTick() - last_stats_ms) >= 1000u)
{
    uint32_t now = HAL_GetTick();
    last_stats_ms = now;

    printf("# stats: rx=%lu dropped=%lu\r\n",
           (unsigned long)rx_count,
```

```c
            (unsigned long)dropped);

    /* Dump every Beacon ID we've ever seen, with current RSSI and the
     * elapsed time since the last reception. Stale entries (older than
     * RSSI_STALE_MS) are flagged so it's clear they would NOT be picked
     * by find_highest_rssi_id().                                        */
    for (int i = 1; i < 256; i++) {
        if (i == BEACON_ID_THREAT) continue;
        if (rssi_table[i].last_seen_ms == 0) continue;
        uint32_t age = now - rssi_table[i].last_seen_ms;
        const char *flag = (age > RSSI_STALE_MS) ? " [STALE]" : "";
        printf("# rssi[ID=0x%02X]=%d dBm age=%lu ms%s\r\n",
               (unsigned)i,
               (int)rssi_table[i].rssi_dbm,
               (unsigned long)age,
               flag);
    }

    /* What would the next PAUSE target be right now? */
    uint8_t cur_id;
    if (find_highest_rssi_id(&cur_id) == 0) {
        printf("# target: ID=0x%02X RSSI=%d dBm\r\n",
               (unsigned)cur_id,
               (int)rssi_table[cur_id].rssi_dbm);
    } else {
        printf("# target: none (no fresh observations)\r\n");
    }
}
```

Listing 4: Place at the bottom of the main `while(1)` loop, after the PAUSE TX scheduler block. Replaces any earlier 10-second `# stats` block.

Expected USART2 output during a healthy ping-pong run:

```
# stats: rx=42 dropped=0
# rssi[ID=0x01]=-87 dBm age=210 ms
# rssi[ID=0x02]=-65 dBm age=98 ms
# target: ID=0x02 RSSI=-65 dBm
```

The three line types tell you, in order: how many packets the ISR has captured and how many the main loop has dropped on overflow; one line per Beacon ID that has ever been heard, with its most recent RSSI and the age of that observation in milliseconds (`[STALE]` when `age > RSSI_STALE_MS`, meaning that ID is excluded from the highest-RSSI lookup); and the unconditional result of `find_highest_rssi_id()` — i.e., which ID the Threat *would* jam if you pressed the button right now. Watching the `# target:` line flip from `0x01` to `0x02` as you move Target A away from the Threat is the canonical demonstration that re-acquisition is working; if the line stays at `0x01` or flips to `none`, the per-ID lines below it tell you why (e.g., Target B never transmitted, so its entry was never populated; or both entries aged past `RSSI_STALE_MS`).

Tuning the staleness window

`RSSI_STALE_MS` defaults to 5000 ms. At Target A's 1 Hz DATA cadence and Target B's reactive ACK cadence, that tolerates 5 missed packets in a row before the entry drops out of selection.

If the snapshot shows entries lingering at `[STALE]` longer than you want before re-acquisition, lower the threshold to 2500 ms (2–3 missed packets). Conversely, if a marginal-link target keeps flickering in and out of selection, raise it. Document the chosen value alongside the ring-buffer-size rationale required by the deliverable.

5.7 Step 6 — Frame builder, CRC-8, and UART forwarder

The frame builder runs in main-loop context on each successful `ring_pop()`. It writes into a single static buffer protected by a DMA-busy flag so the buffer is never overwritten while a previous `HAL_UART_Transmit_DMA` is still in flight.

```c
/* CRC-8/ITU: poly=0x07, init=0x00, no reflect, xorout=0x00.
 * Verified against test vector: crc8_itu("123456789", 9) == 0xF4.
 */
static uint8_t crc8_itu(const uint8_t *buf, uint32_t len) {
    uint8_t crc = 0x00u;
    for (uint32_t i = 0; i < len; i++) {
        crc ^= buf[i];
        for (int b = 0; b < 8; b++) {
            crc = (crc & 0x80u) ? (uint8_t)((crc << 1) ^ 0x07u)
                                : (uint8_t)(crc << 1);
        }
    }
    return crc;
}

/* Builds the 43-byte forwarding frame per LH1-G Table on page 2.
 * The embedded radio-packet CRC-16 (already in pkt->raw[30..31]) is
 * copied through unchanged at frame[40..41]; the frame CRC-8 at
 * frame[42] is computed over frame[1..41] (length, metadata, packet).
 */
static void build_forwarding_frame(uint8_t out[43],
                                   const intercepted_pkt_t *pkt)
{
    out[0] = 0x7Eu;                              /* start delimiter */
    out[1] = 0x29u;                              /* length = 41 */

    /* Timestamp, little-endian */
    out[2] = (uint8_t)(pkt->rx_timestamp_ms      );
    out[3] = (uint8_t)(pkt->rx_timestamp_ms >>  8);
    out[4] = (uint8_t)(pkt->rx_timestamp_ms >> 16);
    out[5] = (uint8_t)(pkt->rx_timestamp_ms >> 24);

    out[6] = (uint8_t)pkt->rssi_dbm;             /* int8 two's complement */
    out[7] = (uint8_t)pkt->snr_q025;

    out[8] = (uint8_t)(pkt->freq_error_hz      );
    out[9] = (uint8_t)(pkt->freq_error_hz >>  8);

    /* Embedded 32-byte radio packet (DATA, ACK, or PAUSE) verbatim */
    for (int i = 0; i < 32; i++) out[10 + i] = pkt->raw[i];

```

```
    out[42] = crc8_itu(&out[1], 41);            /* over bytes 1..41 */
}
```

Listing 5: Add inside `/* USER CODE BEGIN 4 */`. CRC-8/ITU is a separate algorithm from LH1-F's CRC-16; do not share code.

The DMA busy flag pattern protects the 43-byte transmit buffer:

```
/* --- USER CODE BEGIN PV --- */
static uint8_t  tx_frame[43];
volatile uint8_t tx_busy = 0;
/* --- USER CODE END PV --- */

/* --- USER CODE BEGIN 4 --- */
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART3) tx_busy = 0;
}
/* --- USER CODE END 4 --- */
```

Listing 6: Add to `/* USER CODE BEGIN PV */` (storage) and `/* USER CODE BEGIN 4 */` (callback).

Then the main-loop drain (already shown in Step 5) becomes:

```
intercepted_pkt_t pkt;
while (ring_pop(&pkt)) {
    /* Wait for any prior DMA to complete (with a short timeout to avoid
     * deadlock if USART3 wedges).  At 115200 baud, 43 bytes = ~3.7 ms. */
    uint32_t deadline = ms_count + 10u;
    while (tx_busy && ms_count < deadline) { /* spin */ }
    if (tx_busy) { dropped++; continue; }      /* give up; record drop */

    build_forwarding_frame(tx_frame, &pkt);
    tx_busy = 1;
    HAL_UART_Transmit_DMA(&huart3, tx_frame, 43);

    uint8_t pkt_type = pkt.raw[6] & 0x03u;
    printf("RX sender=%02X type=%u rssi=%d snr=%d\r\n",
           pkt.raw[2], pkt_type, (int)pkt.rssi_dbm,
           (int)(pkt.snr_q025 / 4));
}
```

Listing 7: Replace the placeholder `HAL_UART_Transmit_DMA` call inside the Step 5 ring-drain block with this gated version.

Do not reuse LH1-F's `crc16_ccitt` for the frame CRC

The *embedded radio-packet* CRC at bytes 30–31 of the radio packet is CRC-16/CCITT-FALSE, computed by `crc16_ccitt` inside `build_packet` (LH1-F). The *forwarding frame* CRC at byte 42 of the 43-byte UART frame is CRC-8/ITU, computed by `crc8_itu` above. They are different polynomials, different widths, and protect different things. The Module 2 `pkt_parser` checks both independently — mixing them up will cause silent CRC failures on the FPGA.

5.8 Step 7 — Behavioral validation via the Threat's USART2 console

You will run the full system end-to-end and watch the Threat's USART2 console (the ST-Link COM port that already exists on every Nucleo) for evidence that the radio chain, the RSSI table, and the PAUSE TX scheduler are all behaving. This step needs no extra hardware — the `USART3 TX` byte stream is consumed by the FPGA starting in LH1-H and captured to a `.bin` file in LH2-A, so there is no need for a separate USB-to-UART adapter here.

1. Power up Target A and Target B from LH1-F. On the Threat's USART2 console, the 1 Hz snapshot from Step 5b should show both `# rssi[ID=0x01]` and `# rssi[ID=0x02]` lines with sub-second `age=` fields, plus a `# target:` line naming whichever Target is currently louder.

2. Hold the Threat user button (B1) for 5 s. The Threat's USART2 log should print `PAUSE -> ID=...` lines at ∼10 Hz cadence with the chosen ID matching the most recent `# target:` line. The targeted Target's USART2 should also print `"I'm Jammed"` within 100 ms of the press (co-validated with LH1-F).

3. Re-acquisition test: with both Targets running, physically separate Target A from the Threat (or attenuate it with your hand) until the `# target:` line flips to `0x02`. With the button held, the next `PAUSE -> ID=...` entries should address `0x02` and Target B's USART2 should print `"I'm Jammed"` (and Target A's should not). Bring Target A back close and confirm the `# target:` line flips back to `0x01` within `RSSI_STALE_MS` of the last `0x02` reception.

4. Capture a ∼30 s slice of the Threat USART2 log spanning the Target A removal as `LH1G_target_handoff.log`.

Where frame-level CRC validation moved

Earlier revisions of this handout shipped a host-side Python parser that opened a USB-to-UART adapter on the Threat's `USART3 TX` pin, decoded each 43-byte frame, and verified both CRCs. The parser, its tool folder, and the adapter wiring are all retired: the same byte stream is captured in LH2-A using the Nexys A7's onboard USB-UART, and the Module 2 testbenches (LH2-B regression, LH2-D parser, LH2-E CRC) verify framing and CRCs in hardware against those captures. There is nothing left for you to do on the host side in this handout beyond watching the USART2 console.

6 Verification Checklist

- ☐ `RxDone` interrupt fires on every received packet (ISR entry count matches expected receptions).
- ☐ Ring buffer overflows only if you deliberately stall the main loop.
- ☐ 43-byte frames appear on `USART3 TX` for every received DATA, ACK, and PAUSE.
- ☐ RSSI table populated with both Targets' entries within 1 s of starting the ping-pong source.
- ☐ 1 Hz snapshot (Step 5b) prints `# stats`, one `# rssi[ID=0xNN]` line per ever-seen Beacon ID with current RSSI and `age=`, and a `# target:` line; entries past `RSSI_STALE_MS` are flagged `[STALE]`.
- ☐ Re-acquisition test: `# target:` line flips from `0x01` to `0x02` (or back) within `RSSI_STALE_MS` when one Target is removed, and the very next button-held PAUSE addresses the new target.
- ☐ Button-held PAUSE TX: USART2 log shows `PAUSE -> ID=...` at ∼10 Hz with the higher-RSSI Target ID.
- ☐ Co-validated with LH1-F: targeted Target prints `"I'm Jammed"` on its USART2 within 100 ms of button press.
- ☐ Dropped-packet counter remains zero during a 5-minute run.

Concurrency Hazards

- Access `ring_head`/`ring_tail` as `volatile`. Never read the same index twice in the consumer without rechecking.
- Do not call Hardware Abstraction Layer (HAL) `printf` from inside the ISR — it blocks on UART transmit and will drop RF packets.
- Do not overwrite the DMA source buffer while a prior transmission is still in flight. Use a simple busy flag or a second buffer.
- The PAUSE TX scheduler temporarily switches the radio out of `RXCONTINUOUS`; any DATA/ACK arriving during that ∼50 ms TX window is missed. This is the documented half-duplex cost and is the right behavior — it is the reason the framework records the PAUSE-active period as a measurement consideration in Module 5 rather than treating the missed receptions as a bug.

Lab Handout 1-G Deliverable

1. Full Threat firmware source: ring buffer, ISR, RSSI table, PAUSE TX scheduler, and frame builder.
2. Threat USART2 log of a 10-second button-hold session showing ∼100 `PAUSE -> ID=...` entries with the chosen ID matching the configured higher-RSSI Target.
3. Target-handoff log (`LH1G_target_handoff.log`) captured during Step 7's re-acquisition test, showing the `# target:` line transitioning between Beacon IDs as one Target is moved out of range and back, plus the corresponding `PAUSE -> ID=...` addressing change.

4. One-paragraph rationale for the ring buffer size and the RSSI staleness threshold you chose (default 5 s; tune per Step 5b's tcolorbox), including a worst-case arrival-rate analysis.

Glossary

CRC	Cyclic Redundancy Check	2, 3, 4, 15, 19, 20
FPGA	Field-Programmable Gate Array	1, 2, 15, 18
HAL	Hardware Abstraction Layer	19
ISR	Interrupt Service Routine	2, 3, 4, 6, 8, 9, 12, 19
RF	Radio Frequency	1, 19
RSSI	Received Signal Strength Indicator	1, 2, 3, 4, 11, 12, 15, 19, 20
SNR	Signal-to-Noise Ratio	2, 3, 15
UART	Universal Asynchronous Receiver/Transmitter	3, 8